

STORECOPS

GROWTH PLATFORM

Complete Technical Reference

AI-Driven E-Commerce Growth Platform

Architecture · Features · API · Database · Deployment

188 Tests · 58 Source Files · 42 Collections · 6-Layer Architecture · 19 Test Files

Version 1.0.0 | August 2026 | Node.js 18+

storecops-growth-platform

Table of Contents

1. Vision & Product Summary
2. Tech Stack
3. Architecture — 6-Layer Growth Loop
4. Directory Structure
5. Database Collections (42 total)
6. API Endpoints
7. Key Features (Built & Working)
8. Growth Loop (Layer 6)
9. Frontend Architecture
10. Testing
11. Configuration (Environment Variables)
12. Deployment
13. Code Conventions
14. Extension Points
15. Current Status

Purpose: This document is the single source of truth for the Storecops codebase. It is structured for AI-assisted development — paste this into any AI coding assistant to get immediate, accurate help with future development.Đ

1. Vision & Product Summary

****Storecops**** is an AI-driven SaaS growth platform for e-commerce stores (Shopify, WooCommerce, custom). It unifies customer intelligence, competitive monitoring, market trend analysis, automated messaging, SEO optimization, and revenue generation into a single 6-layer system.Đ

****Core value proposition****: A store owner connects their store once !' Storecops automatically monitors competitors, recovers abandoned carts, optimizes pricing, generates SEO fixes, forecasts demand, and sends recovery messages — all without manual intervention.Đ

****Target users****: Shopify/WooCommerce store owners who want automated growth tools without juggling 10 separate apps.Đ

****Business model****: Tiered SaaS subscription — Starter (\$29/mo), Growth (\$79/mo), Enterprise (\$199/mo). Lead generation via free public audit tool that emails branded PDF reports.Đ

2. Tech Stack

Component	Technology
Runtime	Node.js "e 18 (zero TypeScript, pure JavaScript)
Framework	Express.js 4.x
Database	SQLite (via Node.js experimental `node:sqlite`) for persistence; in-memory Map for tests
PDF Generation	PDFKit
Testing	Node.js built-in test runner (`node --test`) — NOT vitest, NOT jest
Shell	PowerShell (Windows) — use `` not `&&`
Deployment	Docker (Alpine), Railway-ready with persistent volumes
Frontend	Vanilla JS (no React/Vue), Chart.js for visualizations
External APIs	Meta Ad Library, Resend (email), Meta WhatsApp Cloud API, Shopify Admin API

****Dependencies****: Only 2 production dependencies — `express` and `pdfkit`. Everything else uses Node.js built-ins.ð

3. Architecture — 6-Layer Growth Loop

The platform follows a strict 6-layer architecture. Each layer only depends on layers below it. Layer 6 (the growth loop) is not a separate module — it's the wiring in `src/platform.js` that closes the loop.ð

```
Layer 6: Growth Loop (platform.js – orchestrates all layers)␣
!␣␣
Layer 5: Reporting (attribution, reports, live orders)␣
!␣␣
Layer 4: Execution (messaging, billing, consent, monitoring)␣
!␣␣
Layer 3: Decision (rules, pricing, campaigns, segmentation)␣
!␣␣
Layer 2: Intelligence (SEO, competitor intel, forecasting, churn, recommendations)␣
!␣␣
Layer 1: Data Foundation (events, customers, inventory, competitors, signals)␣
**Data flow**: Layer 1 ingests raw data !␣ Layer 2 analyzes it !␣ Layer 3 decides what to do !␣ Layer 4 executes actions !␣
Layer 5 measures results !␣ Layer 6 loops back.␣
```

4. Directory Structure

```

storecops-platform/ð
% % % server.js # Entry point – boots platform, starts HTTP serverð
% % % package.json # Dependencies: express, pdfkitð
% % % Dockerfile # Node 20 Alpine, production deployð
% % % Procfile # web: node server.jsð
% % % railway.json # Railway deployment configð
% % % .env.production # Production env templateð
% ð
% % % src/ð
% % % % platform.js # COMPOSITION ROOT – wires all layers togetherð
% % % % config/config.js # All configuration from env varsð
% % ð
% % % % layers/ð
% % % % % data/ # Layer 1: Data Foundationð
% % % % % % customerProfile.js # Unified customer profilesð
% % % % % % eventTracker.js # Append-only event ingestionð
% % % % % % competitorIngestor.js # Competitor data storageð
% % % % % % externalSignals.js # Market signals (trends, news)ð
% % % % % % sentimentCollector.js # Brand sentiment samplingð
% % % % % % inventoryLedger.js # Stock movements (purchase, sell, adjust)ð
% % % % % % searchConsole.js # Google Search Console datað
% % % % ð
% % % % % intelligence/ # Layer 2: Intelligenceð
% % % % % % recommendationEngine.js # Product recommendations (co-view + popularity)ð
% % % % % % churnScoring.js # Customer churn risk scoringð
% % % % % % channelOptimizer.js # Best messaging channel per customerð
% % % % % % inventoryIntelligence.js # Stockout predictions, dead stock alertsð
% % % % % % seoAuditEngine.js # Single-page SEO audit (public /audit)ð
% % % % % % seoGrowth.js # Search Console intent gap, content ideasð
% % % % % % seoAutoFix.js # One-click SEO fix generator (HTML + Liquid)ð
% % % % % % deepAudit.js # Multi-page deep audit (30+ params, 5 categories)ð
% % % % % % trendIntelligence.js # Market trend detection + campaign draftingð
% % % % % % competitorIntelligence.js # Price drop/promo detection across competitorsð
% % % % % % competitorScraper.js # Shopify product scraper (products.json)ð
% % % % % % demandForecast.js # 7-day demand forecastingð
% % % % % % brandSentiment.js # Brand health scoringð
% % % % % % productInsights.js # Fast/slow/dead stock analysisð
% % % % % % defectionDetector.js # Competitor-browsing customer detectionð
% % % % % % seasonalAlerts.js # Upcoming retail moment alertsð
% % % % % % adIntelligence.js # Ad spend/strategy analysisð
% % % % % % metaAdLibrary.js # Meta Ad Library scraperð
% % % % ð
% % % % % decision/ # Layer 3: Decisionð
% % % % % % rulesEngine.js # Custom trigger!'action rulesð
% % % % % % personalization.js # Per-customer content personalizationð
% % % % % % dynamicPricing.js # Competitor-aware price recommendationsð
% % % % % % orchestrator.js # Event!'decision routing, cooldown dedupð
% % % % % % segmentation.js # Customer segmentation (VIP, new, at-risk)ð
% % % % % % campaignGenerator.js # Auto-draft campaigns from trendsð
% % % % % % sendTimeOptimizer.js # Best time to send per customerð
% % % % ð
% % % % % execution/ # Layer 4: Executionð
% % % % % % providers.js # Multi-provider registry (email, WhatsApp, push)ð
% % % % % % emailService.js # Pluggable email (console/resend/smtp) + attachmentsð
% % % % % % whatsappService.js # Meta WhatsApp Cloud API + webhook parsingð
% % % % % % executionService.js # Action queue processorð
% % % % % % websiteBot.js # FAQ/intent-answering chatbotð
% % % % % % retargeting.js # Cart/browse abandoner audience builderð
% % % % % % purchaseOrders.js # Auto-generated supplier POsð
% % % % % % consentService.js # GDPR/CCPA consent trackingð
% % % % % % billingService.js # Shopify Billing integrationð
% % % % % % monitoringService.js # Failure detection + alertingð
% % % % % % pdfService.js # Branded PDF report generator (PDFKit)ð
% % % % ð
% % % % % reporting/ # Layer 5: Reportingð
% % % % % % attribution.js # Revenue attribution per action/campaignð
% % % % % % reportingService.js # ROI, maturity score, weekly digestð
% % % % % % liveOrders.js # Real-time order feed (SSE)ð
% % % ð
% % % % % server/ # HTTP layerð
% % % % % % createApp.js # Express app factory – middleware, routing, public routesð
% % % % % % apiRoutes.js # All /api/v1/* endpoints (behind API key)ð
% % % % % % auth.js # Signup, login, session management, API keysð
% % % % % % security.js # Rate limiter, RBAC, webhook HMAC, GDPR export/deleteð
% % % % % % storeAudit.js # Public site audit (13 checks, no signup)ð
% % % % % % integrations.js # Store connection (Shopify snippet, webhook, CSV)ð

```

```

% % % % % oauthConnectors.js # One-click OAuth connect (Shopify, WooCommerce)ð
% % % % % demoSeed.js # Demo data seeder for dashboard testingð
% % % % % secretRotation.js # Secret fingerprint lifecycle trackingð
% % ð
% % % % % storage/ð
% % % % % store.js # In-memory collection factory (Map-based CRUD)ð
% % % % % sqliteStore.js # SQLite adapter (same interface as in-memory)ð
% ð
% % % public/ # Frontend (vanilla JS, no framework)ð
% % % % % index.html # Landing page (marketing, pricing, features)ð
% % % % % app.html # Main dashboard (SPA-like, tab-based)ð
% % % % % audit.html # Public audit page (free, no signup)ð
% % % % % admin.html # Admin consoleð
% % % % % privacy.html # Privacy policyð
% % % % % terms.html # Terms of serviceð
% % % % % support.html # Support centerð
% % % % % tracker-disclosure.html # Tracker data disclosure (Shopify compliance)ð
% % % % % tracker.js # Client-side tracking snippet (served to stores)ð
% % % % % favicon.svg # Shield iconð
% % % % % js/ð
% % % % % api.js # Tiny API client (session in localStorage)ð
% % % % % app.js # Main dashboard logic (tabs, charts, SEO optimizer)ð
% % % % % audit.js # Audit page (deep audit, gated report, PDF download)ð
% % % % % landing.js # Landing page interactionsð
% % % % % styles/ð
% % % % % app.css # Dashboard stylesð
% % % % % landing.css # Landing page stylesð
% % % % % vendor/ð
% % % % % chart.umd.min.js # Chart.js (CDN-free)ð
% ð
% % % test/ # 19 test files, 188 testsð
% % % % % api.test.js # HTTP API integration testsð
% % % % % audit.test.js # Public site audit testsð
% % % % % auth.test.js # Auth service + HTTP auth testsð
% % % % % competitorScraper.test.js # Shopify scraper + Meta Ad Library testsð
% % % % % connect.test.js # Store connection flow testsð
% % % % % deepAudit.test.js # Deep audit engine + PDF service testsð
% % % % % integrations.test.js # Integration snippet + CSV + webhook testsð
% % % % % layers.test.js # All 6 layers unit testsð
% % % % % liveMonitoring.test.js # Live orders SSE + insights testsð
% % % % % messagingRegression.test.js # Consent, suppression, dedup testsð
% % % % % persistence.test.js # SQLite roundtrip + restart survivalð
% % % % % pipeline.test.js # End-to-end pipeline testsð
% % % % % privacyRegression.test.js # GDPR export/delete/redact testsð
% % % % % queueShutdown.test.js # Consent revocation + uninstall testsð
% % % % % securityRegression.test.js # Tenant isolation, rate limit, HMAC testsð
% % % % % seoOptimizer.test.js # SEO auto-fix + AI optimization testsð
% % % % % toolkit.test.js # Cross-layer feature testsð
% % % % % webhook.test.js # Webhook idempotency testsð
% % % % % whatsapp.test.js # WhatsApp provider + webhook testsð

```

5. Database Collections (42 total)

All collections use the same CRUD interface: ``insert(doc)``, ``findById(id)``, ``find(query)``, ``update(id, patch)``, ``remove(id)``.ð

Layer 1: Data Foundation

Collection	Purpose
`events`	Append-only event log (page_view, add_to_cart, purchase, etc.)
`customers`	Unified customer profiles (merged by email/phone)
`competitorSnapshots`	Competitor product/price snapshots
`externalSignals`	Market trend signals
`sentimentSamples`	Brand sentiment samples
`inventory`	Stock ledger (movements, not just current state)
`searchConsole`	Google Search Console performance data
`competitorAds`	Meta Ad Library snapshots
`trackedCompetitors`	Competitor configs (URLs, page IDs, scrape status)

Layer 2: Intelligence

Collection	Purpose
seoAudits	Single-page SEO audit results
seoOptimizations	Generated SEO + AI fix packages
trendReports	Market trend reports
forecasts	Demand forecast data

Layer 3: Decision

Collection	Purpose
rules	Custom trigger!action rules
actions	Generated actions (messages, price changes)
campaigns	Campaign drafts

Layer 4: Execution

Collection	Purpose
deliveries	Message delivery records
purchaseOrders	Supplier purchase orders

`retargetingAudiences`

Cart/browse abandoner audiences

Layer 5: Reporting

Collection

Purpose

`attributions`

Revenue attribution records

`reports`

Generated reports

Security & Admin

Collection

Purpose

`users`

RBAC accounts (signup users)

`auditLog`

Immutable admin action log

`sessions`

Bearer-token login sessions

Store Connections

Collection	Purpose
`integrations`	Connected stores (Shopify/Woo/webhook/CSV)
`siteAudits`	Free public store audit reports
`connectors`	OAuth app credentials
`oauthStates`	In-flight OAuth CSRF states
`pendingConnections`	Pre-signup store connections

Compliance

Collection	Purpose
`consentRecords`	Customer consent categories per installation
`channelSuppressions`	Per-channel opt-out
`emailSuppressions`	Global do-not-send list

Billing & Monitoring

Collection	Purpose
`subscriptions`	Shopify Billing records
`monitoringEvents`	Failure events (webhook, worker, token, message)
`secretLedger`	Secret fingerprint lifecycle

Deep Audit & Reports

Collection	Purpose
`deepAudits`	Comprehensive multi-page store audits
`reportRequests`	PDF report generation & delivery tracking

6. API Endpoints

Public Routes (no auth required)

Method	Path	
GET	<code>`/health`</code>	Health check
POST	<code>`/api/v1/auth/signup`</code>	Create account
POST	<code>`/api/v1/auth/login`</code>	Login (returns bearer token)
POST	<code>`/api/v1/audit/site`</code>	Free 13-check site audit
GET	<code>`/api/v1/audit/recent`</code>	Recent audit reports
GET	<code>`/api/v1/audit/site/:id`</code>	Get specific audit
POST	<code>`/api/v1/audit/deep`</code>	Deep audit (30+ params, summary only)
GET	<code>`/api/v1/audit/report/:id`</code>	Full deep audit report (auth header checked inline)
GET	<code>`/api/v1/audit/report/:id/pdf`</code>	Download branded PDF (auth header checked inline)

POST

``/api/v1/audit/report/:id/email``

Email PDF report (auth header checked inline)

POST

``/api/v1/track``

Event ingestion (write-only ingest key)

GET

``/api/v1/connect/status``

Connection status per platform

POST

``/api/v1/connect/:platform/start``

Start OAuth connect

GET

``/api/v1/connect/pending/:token``

Check pending connection

Authenticated Routes (API key or bearer token)

All routes under ``/api/v1`` require ``X-API-Key`` or ``Authorization: Bearer <token>``.

Key endpoints:

Method		
GET	<code>`/dashboard/:store_id`</code>	Full dashboard data
POST	<code>`/events/track`</code>	Track customer event
GET	<code>`/customers/:store_id`</code>	List customers
GET	<code>`/recommendations/:store_id`</code>	Product recommendations
GET	<code>`/churn/:store_id`</code>	Churn scores
POST	<code>`/seo/audit`</code>	SEO audit (single page)
POST	<code>`/seo/optimize`</code>	Full SEO optimization
POST	<code>`/seo/one-click-fix`</code>	One-click audit + fix
GET	<code>`/seo/store-info/:store_id`</code>	Auto-detect connected store

POST	/seo/ai-optimize`	AI search optimization only
GET	/competitors/:store_id`	List competitors
POST	/competitors/:store_id`	Add competitor
GET	/inventory/:store_id/analyze`	Inventory intelligence
POST	/inventory/:store_id/purchase`	Record purchase
GET	/orders/:store_id/live`	Live order feed
POST	/rules/:store_id`	Create automation rule
POST	/campaigns/:store_id/generate`	Generate campaign
GET	/attribution/:store_id`	Revenue attribution
GET	/reporting/:store_id`	Full report (ROI, maturity)

7. Key Features (Built & Working)

7.1 Deep Audit & Branded PDF Reports

- Public audit page (`/audit`): Anyone enters a store URL !' gets a summary score (grade A-F), category breakdowns (SEO, Performance, Security, Crawlability, AI Readiness), and top 3 issues
- Deep crawl: Fetches homepage, analyzes SEO (title, meta, H1, OG, JSON-LD, alt text, links), performance (TTFB, page weight, compression), security (HTTPS, 6 security headers), crawlability (robots.txt, sitemap, favicon, llms.txt), AI readiness (6 signals)
- Gated full report: Authenticated users see all checks, detailed findings
- PDF download: Multi-page branded PDF with Storecops branding, score rings, category bars, issue list, and pricing plans page (Starter \$29, Growth \$79, Enterprise \$199)
- Email delivery: PDF attached to branded HTML email with score summary and CTA

7.2 One-Click SEO Optimization

- Connected store auto-detected (no manual URL entry)
- Single button: "Analyze & Fix Everything"
- Generates ready-to-apply code snippets (HTML meta tags, Shopify Liquid templates, Nginx configs)
- Covers: title, meta description, OpenGraph, JSON-LD schemas, robots.txt, security headers, viewport, heading structure, image alt text
- AI search optimization: llms.txt, FAQPage schema, entity markup (Organization/Store with sameAs), HowTo schema, BreadcrumbList schema

7.3 Competitor Intelligence

- Auto-scrape competitor Shopify stores via `/products.json`
- Meta Ad Library integration for ad monitoring
- Price drop and promotion detection
- Competitive positioning analysis

7.4 Customer Intelligence

- Unified profiles (merged by email/phone)
- Churn scoring (inactive customer detection)
- Defection detection (competitor-browsing customers)
- Segmentation (VIP, new, at-risk, browse abandoners)
- Product recommendations (co-view + popularity fallback)

7.5 Automated Execution

- Cart recovery (email + WhatsApp)
- Win-back campaigns (gone customers)
- Browse abandonment messages
- Dynamic pricing recommendations
- Purchase order generation
- Consent-aware messaging (GDPR/CCPA)

7.6 Store Connection

- Shopify: OAuth + Script Tag + Webhooks + Billing
- WooCommerce: REST API key exchange
- Custom: Webhook + tracking snippet
- CSV: Manual inventory import

7.7 Security & Compliance

- Multi-tenant isolation (store A can't read store B data)ð
- HMAC webhook verificationð
- Rate limiting (sliding window)ð
- Secret rotation with fingerprint trackingð
- GDPR export/delete/customer redactð
- PII masking in logsð

8. Growth Loop (Layer 6)

The growth loop is the heartbeat of the platform, implemented in `platform.runGrowthCycle(store_id)`:ð

1. Scan (Layer 2!3): Orchestrator scans store for high-priority eventsð
2. Decide (Layer 3): Rules engine, churn scoring, recommendations fireð
3. Execute (Layer 4): Messages sent, prices adjusted, POs createdð
4. Attribute (Layer 5): Revenue attributed to actionsð
5. Report (Layer 5): ROI calculated, maturity scoredð
6. Loop: Results feed back into Layer 1 as new eventsð

****Schedulers**** (in `server/schedulers.js`):ð

- Growth cycle: periodic per-storeð
- Competitor scrape: every 6 hoursð
- Resync: periodic data refreshð

9. Frontend Architecture

****No framework**** — vanilla JavaScript with a simple tab-based SPA in `app.html`.ð

- `api.js`: Tiny API client. Session stored in `localStorage` as `storecops_session` (JSON: `{ storeId, apiKey, token, email }`). All requests include auth headers.ð
- `app.js`: Main dashboard. Tabs: Overview, Customers, Competitors, Inventory, SEO, Reports, Settings. SEO tab has the one-click optimizer.ð
- `audit.js`: Public audit page. Calls `POST /api/v1/audit/deep` (public), shows summary. If authenticated, fetches full report + enables PDF download + email.ð
- `landing.js`: Landing page interactions.ð

10. Testing

****Command****: `node --test "test/**/*.test.js"` (Node.js built-in test runner — NOT vitest/jest)ð

****188 tests across 19 files****, all passing.ð

****Key conventions****:ð

- Tests use in-memory store (`STORAGE=memory` or `config.env = "test"`)ð
- Test mode bypasses API key middlewareð
- Each test creates its own platform instance for isolationð
- Use unique store IDs per test to prevent cross-test contamination in SQLiteð
- Mock fetch with closure variable pattern for external API testsð
- Integration tests boot a real HTTP server via `createApp()`ð

11. Configuration (Environment Variables)

Variable	Default	Purpose
<code>`NODE_ENV`</code>	<code>`development`</code>	Environment (test/development/production)
<code>`PORT`</code>	<code>`4000`</code>	HTTP port
<code>`API_KEY`</code>	<code>`dev-key`</code>	Master API key
<code>`PUBLIC_URL`</code>	(empty)	Public-facing URL for OAuth callbacks
<code>`STORAGE`</code>	<code>`sqlite` (prod) / `memory` (test)</code>	Storage backend
<code>`SQLITE_PATH`</code>	<code>`data/storecops.db`</code>	SQLite file path
<code>`EMAIL_PROVIDER`</code>	<code>`console`</code>	Email provider (console/resend/smtp)

``WHATSAP
P_PROVID
ER``

``console``

WhatsApp provider (console/meta)

``RESEND_
API_KEY``

`—`

Resend API key (for email)

``WHATSAP
P_ACCESS
_TOKEN``

`—`

Meta WhatsApp token

``WHATSAP
P_PHONE_
NUMBER_I
D``

`—`

Meta WhatsApp phone number

``WEBHOOK
_SECRET``

`—`

HMAC secret for webhooks

``DEFAULT_
STORE_ID``

``store_demo``

Default tenant

``SESSION_
TTL_DAYS``

``7``

Login session TTL

`CHURN_IN
ACTIVE_DA
YS`

`30`

Days before customer considered inactive

`FORECAS
T_WINDOW

`7`

Demand forecast days

`COMPETIT
OR_SCRAP
E_INTERVA
L_HOURS`

`6`

Competitor scrape frequency

`RATE_LIMI
T_WINDOW
_MS`

`60000`

Rate limit window

`RATE_LIMI
T_MAX`

`300`

Max requests per window

`SUBSCRIP
TION_COS
T`

`49`

Monthly cost for ROI calculator

12. Deployment

****Docker****: ``Dockerfile`` (Node 20 Alpine, production deps, health check)ð

****Railway****: ``railway.json`` (Docker build, healthcheck at ``/health``, restart on failure)ð

****Persistent volume****: Mount at ``/app/data`` for SQLite survival across restartsð

****Deploy steps****:ð

1. Push to GitHubð
2. Connect Railway to repoð
3. Add volume at ``/app/data``ð
4. Set env vars: ``NODE_ENV=production``, ``API_KEY=<random>``, ``PUBLIC_URL=<railway-url>``ð
5. Deploy — auto-builds from Dockerfileð

13. Code Conventions

- Pure JavaScript — no TypeScript, no transpilationð
- ``"use strict"`` at top of every fileð
- Factory functions — ``createXxx({ store, config })`` pattern (no classes)ð
- Async/await throughout — no callbacks, no ``.then()``ð
- Error handling — throw Error with descriptive message, caught by ``wrap()`` in routesð
- Logging — ``console.log`` with ``[TAG]`` prefix. PII masked (email: ``j*@g*``).ð
- No external dependencies beyond express + pdfkit — use Node.js built-insð
- Test command: ``node --test "test/**/*.test.js"`` (PowerShell: use ``;`` not ``&&``)ð

14. Extension Points

When adding new features, follow these patterns:ð

1. New intelligence module: Create ``src/layers/intelligence/myModule.js`` ``!' export`` ``createMyModule({ store, ...deps })`` ``!' wire in`` ``platform.js``ð
2. New API endpoint: Add to ``src/server/apiRoutes.js`` inside ``createApiRouter(platform)`` using ``wrap(async (req) => { ... })``ð
3. New public endpoint: Add to ``createAuditRouter()`` or create new public router in ``createApp.js`` (mounted BEFORE the keyed API)ð
4. New collection: Add name to ``COLLECTIONS`` array in ``src/storage/store.js``ð
5. New email template: Add method to ``emailService`` (supports HTML + attachments)ð
6. New test file: Create ``test/myFeature.test.js`` — use ``node --test`` compatible assertionsð

15. Current Status

- 188/188 tests passingð
- 58 source files (src/)ð
- 19 test files (test/)ð
- 17 frontend files (public/)ð
- 42 database collectionsð
- Fully deployable to Railway/Dockerð
- Production-ready architecture with multi-tenant isolation, consent management, GDPR compliance, secret rotation, and monitoringð

